

Algorithmes et Pensée Computationnelle

Algorithmes de recherche - Exercices de base

Le but de cette séance est de se familiariser avec les algorithmes de recherche. Dans cette série d'exercices, nous manipulerons des listes et collections en Java et Python. Nous reviendrons sur la notion de récursivité et découvrirons les arbres de recherche. Au terme de cette séance, l'étudiant sera en mesure d'effectuer des recherches de façon efficiente sur un ensemble de données.

Le code présenté dans les énoncés se trouve sur Moodle, dans le dossier Code.

1 Recherche séquentielle (ou recherche linéaire)

1.1 Définition

Une recherche **séquentielle** (ou **linéaire**) est une méthode permettant de trouver un élément dans un ensemble de données (liste, tableau ou dictionnaire). Elle vérifie un à un chaque élément de gauche à droite jusqu'à ce qu'une correspondance soit trouvée ou que toute la liste ait été parcourue.

Si l'élément recherché est trouvé, l'algorithme **renvoie l'index**, c'est-à-dire la position, de l'élément dans l'ensemble de données.

Sa complexité dans le pire des cas est de $O(n)$ correspondant à la longueur de l'ensemble de données, et dans le meilleur des cas de $O(1)$, lorsque l'élément se trouve en première position. Dans la pratique, l'algorithme de recherche séquentielle n'est pas couramment utilisé eu égard à sa complexité élevée et des alternatives de recherche plus efficaces comme la recherche binaire.

1.2 Exercices

Question 1: (🕒 10 minutes) Recherche séquentielle - 1 (Python)

1. À partir des éléments ci-dessous, écrivez une fonction qui cherche un élément e dans la liste L .

La fonction doit retourner l'index de l'élément correspondant de la liste si e est dans la liste et -1 si e n'est pas dans la liste (avec $e = 100$).

Python :

```
1 def recherche_sequentielle(liste, e):
2     # complétez ici
3
4 if __name__ == '__main__':
5     L = [3, 55, 6, 8, 3, 5, 56, 33, 6, 5, 3, 2, 99,
6          53, 532, 75, 21, 963, 100, 445, 56, 56, 24]
7     e = 100
8
9     resultat = recherche_sequentielle(L, e)
10    print(resultat)
```

Conseil

Définissez votre fonction de recherche séquentielle en utilisant une boucle `for` ou une boucle `while`.

Attention : la fonction doit retourner l'index de la valeur et non la valeur elle-même. Pour cela :

- Avec une boucle `for`, pensez à utiliser `range(len(liste))`.
- Avec une boucle `while`, utilisez une variable d'incrément `i = i + 1`.

Utilisez la fonction `print()` pour afficher l'index retourné par la fonction de recherche. Si la valeur retournée est -1, affichez un autre message indiquant que l'élément n'était pas dans la liste.

>_ Solution

Python :

```
1 # Définition de la fonction
2 def recherche_sequentielle(liste, e):
3     for idx in range(len(liste)): # idx représente l'index
4         if liste[idx] == e:
5             return idx
6
7     return -1
8
9
10 if __name__ == '__main__':
11     # Déclaration de la liste et de la variable x
12     L = [3, 55, 6, 8, 3, 5, 56, 33, 6, 5, 3, 2, 99,
13          53, 532, 75, 21, 963, 100, 445, 56, 56, 24]
14     e = 100
15
16     # Appel de la fonction
17     resultat = recherche_sequentielle(L, e)
18     print(resultat)
```

2. Re-faire la question en utilisant la fonction `index` en Python.

Conseil

La fonction `index` est appelée sur une liste `L` afin de déterminer l'index d'un élément `e` : `L.index(e)`.

- Si `e` n'est pas dans la liste `L` : la fonction lève une exception de type `ValueError`.
- Si `e` est dans la liste `L` : la fonction retourne l'index de la première occurrence de `e`.

>_ Exemple

```
1 if __name__ == '__main__':
2     L = [123, 321, 328, 472, 549, 328]
3     e = 328
4     resultat = recherche_sequentielle(L, e)
5     print(resultat) # Affiche 2
```

>_ Solution

Python :

```
1 def recherche_sequentielle(liste, e):
2     if e in liste:
3         # L'algorithme prend fin aussitôt que la valeur recherchée est
          trouvée.
4         return liste.index(e)
5     return -1 # Si la valeur n'est pas trouvée, retourne -1
```

>_ Solution

Python :

```
8 # Solution alternative
9 def recherche_sequentielle(liste, e):
10     try:
11         # L'algorithme prend fin aussitôt que la valeur recherchée est
          trouvée.
12         return liste.index(e)
13     except ValueError:
14         return -1
15
16
17 if __name__ == '__main__':
18     L = [123, 321, 328, 472, 549, 328]
19     e = 328
20     resultat = recherche_sequentielle(L, e)
21     print(resultat) # Affiche 2
```

Question 2: (🕒 10 minutes) Recherche séquentielle - 2 (Python)

Soit une liste d'entiers **non triée** L ainsi qu'un entier e. Écrivez un programme qui retourne l'élément de la liste L dont la valeur est la plus proche de e en utilisant une recherche séquentielle.

>_ Exemple

```
1 # Déclaration et initialisation de la liste L et de la variable e
2 L = [16, 2, 25, 8, 12, 31, 2, 56, 58, 63]
3 e = 50
4
5 # Exécution de la fonction
6 resultat = plus_proche_sequentielle(L, e)
7 print(resultat) # affiche 56
```

Au cas où deux éléments se trouveraient à équidistance de e, renvoyez l'élément le plus petit.

Python :

```
1 # Définition de la fonction ayant pour argument une liste et un nombre
2 def plus_proche_sequentielle(liste, nb):
3     # Initialisation de la variable (-1 car les différence calculée après
          seront toujours positives)
4     diff = -1
5     resultat = None # Initialisation de la variable pour le résultat
6
7     # Complétez ici
```

```

8
9
10 if __name__ == '__main__':
11     # Testez votre code ici

```

💡 Conseil

Complétez la fonction `plus_proche_sequentielle` puis exécutez le code.

Pour comparer les écarts, utilisez les valeurs absolues, car la différence la plus petite peut être positive ou négative. En Python, la fonction `abs()` retourne la valeur absolue d'un nombre. Par exemple : `abs(3 - 10)` retourne 7.

Étant donné que la liste n'est **pas triée**, l'algorithme doit nécessairement la parcourir entièrement.

L'algorithme doit calculer la différence entre `e` et chaque élément de la liste `L`, tout en conservant en mémoire la plus petite différence rencontrée. À la fin, il retourne l'élément de la liste correspondant à la plus petite différence.

>_ Solution

Python :

```

1  # Définition de la fonction ayant pour argument une liste et un nombre
2  def plus_proche_sequentielle(liste, nb):
3      # Initialisation de la variable (-1 car les différences calculées
      # après seront toujours positives)
4      diff = -1
5      resultat = None # Initialisation de la variable pour le résultat
6
7      # Solution
8      for elem in liste:
9          if diff == -1 or abs(elem-nb) < diff:
10             diff = abs(elem-nb) # nouvelle diff.
11             resultat = elem
12             elif (abs(elem-nb) == diff):
13                 resultat = min(elem, resultat)
14
15     return resultat
16
17 if __name__ == '__main__':
18     # Déclaration et initialisation de la liste L et de la variable e
19     L = [16, 2, 25, 8, 12, 31, 2, 56, 58, 63]
20     e = 50
21
22     # Exécution de la fonction
23     resultat = plus_proche_sequentielle(L, e)
24     print(resultat) # affiche 56

```

2 Recherche binaire — Binary search

2.1 Définition

Le but de la recherche binaire est de trouver l'élément recherché de façon optimale. Pour cela, il est nécessaire d'utiliser **une liste d'éléments triés**.

La complexité de l'algorithme de recherche binaire est $O(\log n)$. Cependant, il ne faut pas oublier le coût lié à l'obtention d'une liste triée à partir d'une liste non triée.

L'algorithme de recherche binaire divise l'intervalle de recherche par deux à chaque itération jusqu'à ce qu'il trouve l'élément `key` ou que l'intervalle soit vide.

Ainsi, si `key` est plus petit que l'élément du milieu, l'algorithme va choisir la moitié de gauche comme intervalle de recherche et ainsi de suite. Si `key` est plus grand que l'élément du milieu la recherche va se faire dans la moitié droite de l'intervalle.

La recherche binaire se base sur les comparaisons d'ordre, alors que la recherche séquentielle se base les comparaisons d'égalité pour trouver l'élément recherché.

2.2 La récursivité

Une fonction récursive est une fonction qui s'appelle elle-même pendant son exécution. Vous trouverez ci-dessous un exemple de fonction récursive utilisée pour effectuer un calcul factoriel.

(Rappel : $4! = 4 \times 3 \times 2 \times 1 = 24$)

```
1 def factorial(n):
2     if n == 1:
3         return n
4     else:
5         return n * factorial(n - 1)
6
7 # Exécution de la fonction
8 print(factorial(4)) # Affiche 24
```

Les fonctions récursives sont courantes en informatique car elles permettent aux programmeurs d'écrire des programmes efficaces en utilisant une quantité minimale de code. Leur principal inconvénient est le fait qu'elles peuvent provoquer des exécutions infinies et d'autres résultats inattendus si elles ne sont pas écrites correctement. Si la fonction n'inclut pas les cas permettant d'arrêter la récursivité, celle-ci se répétera à l'infini, provoquant le plantage du programme ou, pire encore, l'arrêt de tout le système informatique.

2.3 Exercices

Question 3: (🕒 20 minutes) Algorithme de recherche binaire — Binary search (Python)

À partir de la liste d'éléments **triés** ci-dessous, écrivez premièrement **une fonction récursive** puis **une fonction itérative** qui cherche `key` dans la liste. La fonction doit retourner l'**index** de l'élément correspondant de la liste si `key` est dans la liste et `-1` dans le cas contraire.

💡 Conseil

Testez votre implémentation avec la liste `L=[1, 3, 4, 5, 7, 8, 9, 15]` dans lequel vous cherchez `key=5`.

Question 3.1 - Version récursive

```
1 def binary_search_recursive(key, array, low=0, high=None):
2     # Complétez ici
3
4 if __name__ == '__main__':
5     # Testez votre code ici
```

Question 3.2 - Version itérative

```

1 def binary_search(key, array):
2     # Complétez ici
3
4 if __name__ == '__main__':
5     # Testez votre code ici

```

💡 Conseil

Complétez la fonction récursive `binary_search_recursive` et la fonction itérative `binary_search`.

Détails sur les arguments de la fonction `binary_search_recursive`:

- `L`: La liste dans laquelle nous effectuons la recherche.
- `low`: L'indice de départ de la partie de la liste à fouiller (0 au départ).
- `high`: Le dernier indice de la partie de la liste à fouiller (`len(L) - 1` au départ).
- `key`: La valeur recherchée.

Ainsi, à chaque itération, vos fonctions vont modifier les valeurs de base données en argument pour resserrer l'intervalle jusqu'à trouver la valeur recherchée.

Pour la version itérative, il est conseillé d'utiliser une boucle `while`.

💡 Conseil

Pour définir le milieu d'une liste qui contient un nombre pair ou impair d'éléments, divisez l'ensemble en 2 et utilisez la fonction `int()` pour convertir le résultat en entier.

```

1 liste = [1,2,3,4,5]
2 low = 0
3 high = len(liste) - 1
4 m = int((low+high)/2) # ou (low+high)//2

```

>_ Solution

3.1 - Version récursive

```

1 def binary_search_recursive(key, array, low=0, high=None):
2     if high is None:
3         high = len(array) - 1
4
5     # Base case: not found
6     if low > high:
7         return -1
8
9     mid = (low + high) // 2
10
11     if key < array[mid]:
12         return binary_search_recursive(key, array, low, mid - 1)
13     elif key > array[mid]:
14         return binary_search_recursive(key, array, mid + 1, high)
15     else:
16         return mid
17
18 if __name__ == '__main__':
19     L = [1,3,4,5,7,8,9,15]
20     x = 5
21     idx = binary_search_recursive(x, L)
22     print(idx)

```

>_ Solution

3.2 - Version itérative

```
1 def binary_search(key, array):
2     low = 0
3     high = len(array) - 1
4
5     while low <= high:
6         mid = (low + high) // 2
7
8         if key < array[mid]:
9             high = mid - 1
10        elif key > array[mid]:
11            low = mid + 1
12        else:
13            return mid
14
15    return -1
16
17 if __name__ == '__main__':
18     L = [1, 3, 4, 5, 7, 8, 9, 15]
19     x = 5
20     idx = binary_search(x, L)
21     print(idx)
```

3 Arbre de recherche binaire — Binary search tree

3.1 Définition

Un arbre de recherche binaire est une structure de données au même titre que les listes, tuples, dictionnaires, etc. Leur particularité est qu'au lieu d'ordonner les éléments les uns à la suite des autres, les arbres de recherche binaire enregistrent les valeurs de manière relationnelle.

En effet, l'arbre est divisé en branches qui peuvent elles-même contenir deux branches (enfants) et ainsi de suite. Lorsqu'une branche n'a pas de branches subséquentes (enfants), on parle de feuille.

Les branches peuvent donc contenir de 0 à 2 autres branches. On parle alors de branche de gauche et de celle de droite. La branche de gauche est forcément plus petite que la branche parente et celle de droite est forcément plus grande.

De cette façon, on garantit que l'arbre est toujours ordonné même en rajoutant ou en retirant des éléments

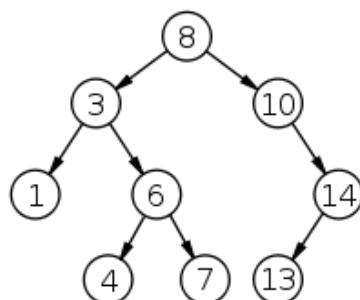


FIGURE 1 – Exemple d'arbre de recherche binaire

3.2 Exercices

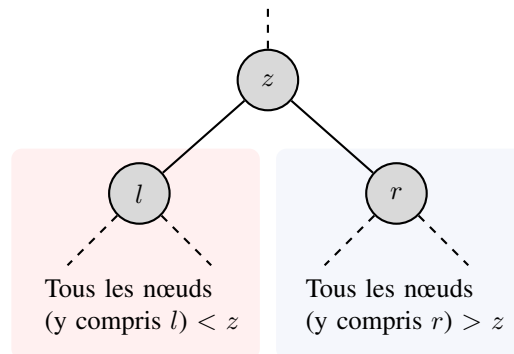
Question 4: (🕒 10 minutes) Propriété des arbres de recherche binaire

Supposons que nous ayons des nombres entre 1 et 1000 dans un arbre binaire de recherche, et que nous voulions rechercher le nombre 363. Lesquelles des séquences suivantes ne pourraient pas être une séquence des nœuds examinés avant de trouver 363 ?

1. 2, 252, 401, 398, 330, 344, 397, 363.
2. 924, 220, 911, 244, 898, 258, 362, 363.
3. 925, 202, 911, 240, 912, 245, 363.
4. 2, 399, 387, 219, 266, 382, 381, 278, 363.
5. 935, 278, 347, 621, 299, 392, 358, 363.

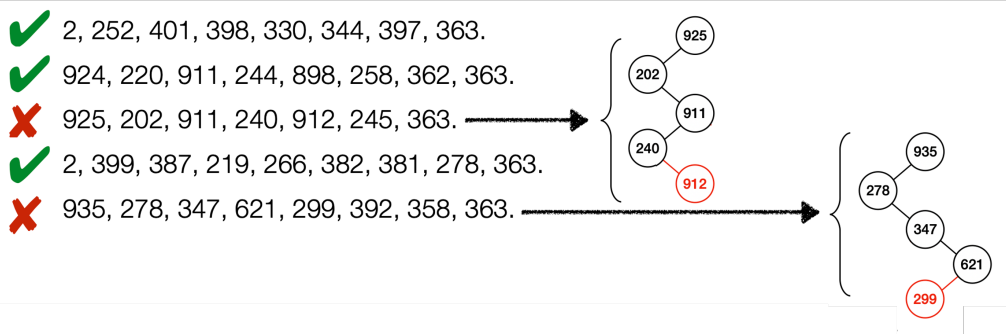
💡 Conseil

Rappelez-vous de la propriété suivante des arbres de recherche binaire :



Tous les nœuds dans le sous-arbre à gauche ont une valeur inférieure à la valeur du nœud z et tous les nœuds dans le sous-arbre à droite ont une valeur supérieure à la valeur du nœud z .

>_ Solution



>_ Solution

- Le troisième choix est faux car 912 se trouve dans le sous-arbre à gauche de 911 ce qui contredit la propriété des arbres de recherche binaire.
- Le cinquième choix est faux car 299 se trouve dans le sous-arbre à droite de 347 ce qui contredit la propriété des arbres de recherche binaire.

Question 5: (🕒 20 minutes) Recherche dans un arbre - Python

Dans l'exercice suivant, nous vous donnons un arbre binaire avec les mêmes valeurs que le schéma précédent et dont la racine est la variable `root`.

Écrivez une fonction qui retourne `True` si la valeur `value` se trouve dans l'arbre et `False` dans le cas contraire. Vous pouvez utiliser la variable `value` pour afficher la valeur d'un nœud (`node`) et les variables `left` et `right` sur les nodes pour accéder aux branches enfants, utilisez `node.value`, `node.left` et `node.right`.

Complétez la fonction `recherche_arbre`.

```
1 import sys
2 import traceback
3
4 def recherche_arbre(node, value):
5     # Complétez ici
6
7
8 #Attention à inclure les lignes de codes additionnelles présentes dans le
   fichier "question6.py" sur Moodle
9
```

Conseil

Indice : Utilisez une fonction récursive.

Pour cette question, vous devez télécharger le fichier **question6.py** sur Moodle et copier tout son contenu dans votre fichier Python. Celui-ci contient les classes permettant de définir les arbres selon le concept vu en cours.

Il contient également un code de “vérification”. Celui-ci va automatiquement tester votre fonction avec différents paramètres et vérifier si les résultats obtenus correspondent aux résultats attendus.

>_ Solution

```
1 import sys
2 import traceback
3
4 def recherche_arbre(node, value):
5     #Solution
6     if node == None:
7         return False
8
9     if node.value == value:
10        return True
11    if node.value > value:
12        return recherche_arbre(node.left, value)
13    else:
14        return recherche_arbre(node.right, value)
```